
FVSpec: Real-World Property-Based Tests as Lean Challenges

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 We present a benchmark for evaluating AI models and agents on real-world formal
2 software verification tasks. We first scrape 11,039 property-based tests (PBTs)
3 from real-world Python repositories, then automatically translate them into Lean 4
4 specifications with `sorry` placeholders. Translating PBTs into Lean specifications
5 is challenging: it requires modeling Python semantics in Lean, inferring the logical
6 property encoded in an imperative PBT, and handling the inherent difficulties
7 of dependently-typed programming in a seldom-used language. We describe a
8 three-agent LLM pipeline for transpiling PBTs into Lean specifications, evaluate
9 coverage and quality metrics, and provide baselines for proof generation using
10 several automated and model-based approaches. Our benchmark aims to drive
11 progress on the underexplored problem of AI-assisted formal verification of real-
12 world software, which is of increasing interest as AI produces more and more of
13 the world’s code.

14 1 Introduction

15 Several AI safety proposals are premised on implementing safeguards by way of *formal verification*
16 (FV)¹ – that is, software systems that can produce and unequivocally certify mathematical proofs.
17 These systems can be split into *interactive theorem provers* (ITPs) which accept and validate user-
18 supplied proofs, and *automated theorem provers* (ATPs) which attempt (but often fail) to prove
19 theorems fully autonomously. ARIA’s Safeguarded AI [12], Guaranteed Safe AI [11], and Scalable
20 Formal Oversight [53, 41], among others, all suggest that in the future, all AI-generated software will
21 be formally verified by autonomous FV agents, who will steer ITPs in much the same way that
22 coding agents currently steer software production.

23 The only way FV can play this role is if agents can conduct verification work at least as well and as
24 quickly as they can produce code. Otherwise, FV becomes a bottleneck and will naturally wither
25 under the capital pressure to iterate². There is currently little evidence that such capabilities are
26 possible. To achieve them, we would first need high-quality benchmarks to train on. Existing
27 benchmarks (see Table 1) are too small (e.g. [34]), focused on advanced mathematics rather than
28 engineering (e.g. [19]), or derived from expert-driven verification projects (as is the case in [9])
29 which are quite different from what you would get were you to verify the average software project.
30 To clarify this last point: historically, FV has been bottle-necked by the number of human expert
31 practitioners able to write proofs in an ITP³, and therefore, outside of pedagogy or academic research,

¹FV forms a subset of *formal methods*, which also include PBT, model checking, and abstract interpretation.

²The reader is referred to [44] for a detailed discussion of how socioeconomic pressures degrade safety cultures and cause predictable accidents.

³We were unable to find any formal estimate of the number of human formal methods practitioners today. However, as a very rough proxy, de Almeida Borges et al. [13] conducted the largest ever survey of users of any particular ITP – in this case

FV was only ever applied to the most mathematically complex and safety-critical applications, such as compilers [31] or an OS kernel [29]. These projects are generally monolithic, closed systems, in contrast to the average software project which may have a frontend and backend, a database, reliance on several external black-box APIs such as an inference provider or a logging service, and an unnecessarily large dependency tree. The latter category of (normal) software is actually in many ways much more challenging to verify because it necessitates reasoning about black-box external systems and the interactions of several different components, potentially written in different languages and operating on different chips [7, 42].

Benchmark	Language	Theorems	Focus
Minif2f [59]	Lean, Isabelle, HOL Light, Metamath	488	math Olympiad
PutnamBench [51]	Lean, Isabelle, Coq/Rocq	640	math Olympiad
Fimo [33]	Lean	149	math Olympiad
DafnyBench [34]	Dafny	782	scraped Dafny proofs
ProofNet [2]	Lean	371	U math
RLM25 [43]	Lean	619	G math
Lean Workbook [57]	Lean	57,000	HS, U, G math
FormalMATH [58]	Lean	5,560	math Olympiad
LEAN-GitHub [56]	Lean	28,597	scraped Lean proofs
ProverBench [45]	Lean	325	math Olympiad
MiniCTX [24]	Lean	762	six Lean projects
LeanGeo [46]	Lean	122	HS, U, G math
IneqMath [36]	Lean	1,522	math Olympiad
FATE-{M,H,X} [27]	Lean	350	U, G math
VeriBench [4]	Lean	857	software verification
FVAPPS [17]	Lean	4,715	software verification (from APPS)
CLEVER [49]	Lean	161	software synthesis & verification
Vericoding [8]	Lean, Dafny, Verus	12,504	software synthesis & verification
MUSTARD [25]	Lean	5,866	HS, U math
FormL4 [35]	Lean	14,510	U math
CoqStoq (Rango) [50]	Coq/Rocq	10,396	software verification
MSC-180 [32]	Lean	180	U, G math
HOList [3]	HOL Light	29,462	U, G math

Table 1: **Prior ITP benchmarks.** HS = high-school, U = undergraduate, G = graduate. If one benchmark improves upon another, we exclude the lesser.

In light of the limitations of prior works, we generate a new benchmark, FVSpec, from public uses of property-based testing (PBT). While FV itself is rarely used in software development, PBT is a “close sibling” technology which has seen significant adoption. In both FV and PBT, engineers write *specifications*: logical properties that the software must obey. For example, we might require that a Python function `isort(1st)` always returns a sorted permutation of its input `1st`. The difference lies in how this property is checked. In FV, the property is proved using mathematical techniques—this results in almost-perfect confidence, but requires proof engineering by highly expert teams [16]. In PBT, the code is randomly tested—e.g. for random values of `x`. This is a push-button process with no proof engineering, and provides only statistical assurance.

PBT is typically much cheaper than FV, and as a result has seen wider adoption. In fact, PBTs are typically seen as the first step toward FV. For example, in the interactive theorem prover ACL2s [15], any unproven theorem statement is automatically treated as a PBT until the author supplies a proof. Thus the idea of translating PBTs to theorems for FV is natural and, to an extent, industry-standard. We build on this. First, we scrape 11,039 PBTs written in Python’s Hypothesis framework [38]. Each PBT in our dataset can be seen as a formal conjecture about a piece of Python code, which the author would like to either substantiate (prove) or disprove. We refer to our scraped PBT corpus as FVSpec:PBT. To make it possible for an AI to resolve these conjectures, we translate both code and

Coq, which is now called Rocq – at 466 participants. As a point of contrast, that year’s survey by the Rust foundation of Rust software engineers had 7,156 participants [48]. Anecdotally, most ITP work today is in Rocq, Lean 4, or Isabelle.

PBTs to the theorem prover Lean, which is becoming the de-facto standard in AI formal verification. We refer to our resultant corpus of Lean translations as FVSpec:FV. Once the Lean version of the conjecture is (eventually) proven, it can be called a *theorem*. Until then, it has a sorry placeholder where the proof belongs. This approach was pioneered by Dougherty and Mehta [17] on the FVAPPS verification benchmark. In FVAPPS, source programs were taken from the APPS benchmark by Hendrycks et al. [22], translated to PBTs, and then lifted to theorems. We apply the same process at scale to PBTs found in the wild.

We use AI agents to translate from Python to Lean, in the style of FVAPPS. This process is necessarily lossy, that is to say, it does not guarantee strict equivalence. However, strict equivalence is only needed if the goal is to aid the authors of the original software by resolving their open conjectures. Although when applicable we consider this goal a positive secondary contribution of our work, our actual motivation is to build a large benchmark of realistic and high-quality software verification exercises, representative of the diversity of code found in “normal” codebases. Thus, we don’t worry whether or not our translations are perfectly equivalent, so long as they are equivalently interesting. The result is 9,415 Lean 4 verification challenges. Each consists of four artifacts (Figure 1): first, the original Python implementation; second, the original Python PBT exercising it; third, a complete Lean implementation of the same function; and fourth, a Lean theorem statement of the property with a sorry for the proof. Unlike existing FV benchmarks, each problem corresponds to real-world software written by engineers with no formal verification experience. As Table 1 shows, the bulk of ITP benchmarks target mathematics rather than software. The software-verification benchmarks that are large—Vericoding [8] (12,504 tasks), LEAN-GitHub [56] (28,597 theorems), and CoqStoq [50] (10,396 theorems)—all derive from code that was written specifically for or in an ITP: Vericoding aggregates problems from competitive verification events (VerifyThis) and the Archive of Formal Proofs; LEAN-GitHub and CoqStoq scrape GitHub repositories where the primary artifact is the Lean or Coq proof. FVAPPS [17], the closest methodological predecessor, starts from competitive programming puzzles (APPS [22]) translated into PBTs, not from code written in the ordinary course of software development. None of these benchmarks contain specifications authored by practicing engineers who had no formal verification goal in mind, which is precisely what ours does—putting our problems out of distribution relative to anything an AI is likely to have memorized.

Our main contributions are:

1. **FVSpec:PBT** — a corpus of 11,039 deduplicated Python property-based tests scraped from 333 open-source repositories, the first large, license-clean PBT dataset drawn from ordinary software development (no competition problems, no curated ITP code).
2. **FVSpec:FV** — 9,415 Lean 4 formal verification challenges lifted from 2,772 of those PBTs by an agentic transpilation pipeline with iterative LSP repair, each annotated with structural faithfulness scores (median 0.65), a calibrated difficulty label (62% hard), and full artifact metadata.
3. **Baseline evaluations** of Claude Sonnet 4.6 and GPT 5.4 showing that both models struggle on the hard subset, confirming the benchmark is far from saturated.
4. **Open-source release** of the full pipeline—scraper, dependency extractor, formalization agent, and post-production scripts—under the MIT license.⁴

The rest of the paper is organized as follows. We formalize the task in Section 2. We characterize our input corpus, detailing how we scraped and deduplicated 11,039 Hypothesis tests from open-source repositories, in Section 3, and discuss how that corpus compares to a similar one released by the Hypothesis team [14]. Then, we describe our generation pipeline—scraping, dependency extraction, LLM-based transpilation with iterative Lean LSP repair, and post-production cleaning—in Section 4. We characterize the benchmark and present baseline results in Section 5, discuss threats to validity in Section 6, survey related work in Section 7, and conclude in Section 8.

2 Task Definition

Each problem in the FVSpec benchmark consists of four components: (1) the original Python implementation of the function under test, (2) the original Python PBT exercising that implementation,

⁴TODO

(3) a full Lean implementation of the same function, and (4) a Lean theorem corresponding to the PBT, with a `sorry` placeholder where the proof should go. The AI’s task is to fill in every theorem `sorry` with a Lean proof term that compiles cleanly under `lake build`; submissions are then scored by whether the proof retains any `sorry`, `admit`, or axioms beyond Mathlib’s defaults. Each problem also includes a URL pointing to the original source code and license information.

As a running example, consider `isort(lst)`. Figure 1 shows all four artifacts of a single FVSpec problem. The left column holds the original implementation (top) and the PBT exercising it (bottom); the right column holds the corresponding `Impl.lean` (top), a complete Lean port of the function, and `Spec.lean` (bottom), containing two theorems—one for sortedness, one for permutation—each with their own `sorry`. The AI’s task is to discharge both Lean `sorry` placeholders.

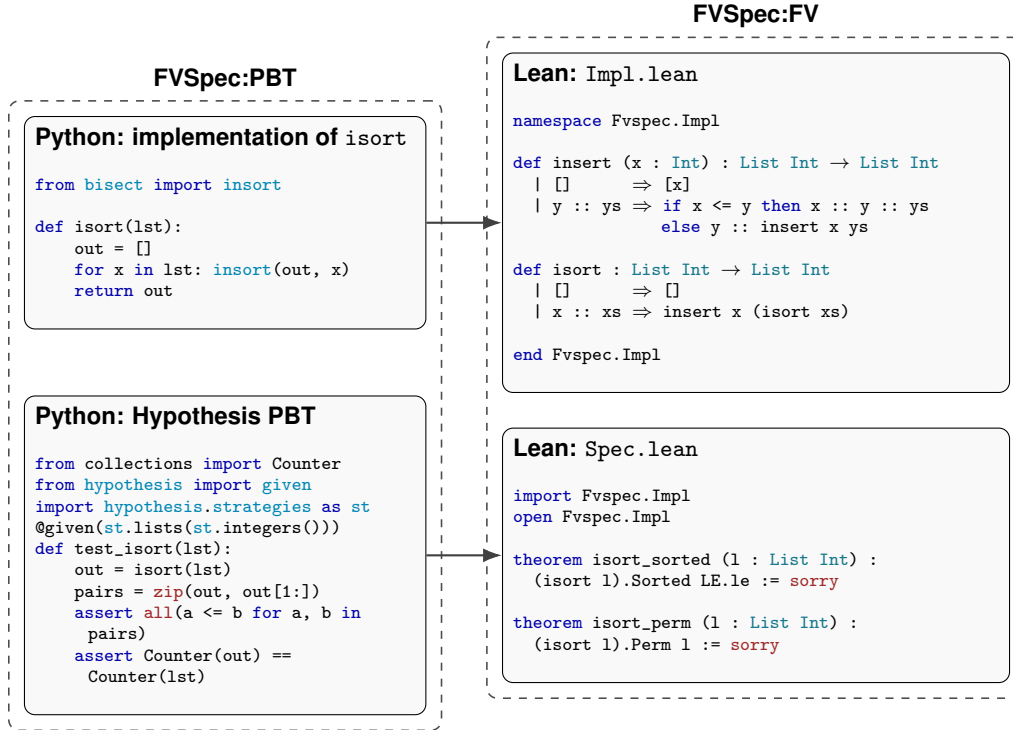


Figure 1: The four artifacts of a single FVSpec problem. Each problem is partitioned into the *PBT corpus* (FVSpec:PBT, left)—the original Python implementation and Hypothesis test we scrape from GitHub—and the *formal-verification challenge* (FVSpec:FV, right) we transpile from it: a complete Lean port of the implementation and a Lean theorem statement of the property under test. The AI must discharge both Lean `sorry` placeholders, one per theorem.

We encounter two practical issues when translating Python PBTs into Lean implementations and (unproven) specifications.

Nonsensical theorems. Our pipeline can emit well-formed but useless statements in three ways.

(i) *Trivial Unit-typed conjectures.* When the formalization agent cannot recover a function’s true return type, it falls back on `Unit` as a placeholder. The resulting theorem is a tautology—e.g. for our running example, `theorem isort_check : isort lst = () := by plausible`, which holds for any total function into `Unit` and so carries no verification value.

(ii) *Ill-typed or unresolved-symbol theorems.* The formalization agent works from the Python source and writes Lean by analogy. When it invents a Lean name that does not exist—for instance, writing `theorem isort_perm (l : List Int) : (isort l).permutationOf l := sorry`, where `permutationOf` is not a member of `List` (the correct name is `List.Perm`)—or attaches a type signature inconsistent with the surrounding code, the Lean elaborator rejects the file with errors like *unknown identifier*, *type mismatch*, *function expected*, or *failed to synthesize*.

131 (iii) *Spec theorems leaking into Impl.lean*. Our pipeline runs two separate agents: one writes
 132 `Impl.lean` (definitions only), the other writes `Spec.lean` (theorems only). The implementa-
 133 tion agent is explicitly instructed to emit no theorems, but occasionally—when shown the `isort`
 134 source—it pre-empts the spec agent by hallucinating a namespace `Fvspec.Spec ... theorem`
 135 `isort_sorted ... end Fvspec.Spec` block inside `Impl.lean`, which violates our file split and
 136 would silently expand the proof obligation if left in.

137 We catch (i) by regex, drop (ii) at compile time, and strip (iii) with a namespace-aware filter. A
 138 theorem that survives all three filters is a useful challenge problem even if it has drifted from the
 139 original PBT—because Lean checks proofs against *the theorem we generated*, not the informal
 140 property.

141 **Lean as a (nearly) perfect oracle.** The Lean kernel arbitrates correctness, which is qualitatively
 142 much stronger than statistical sampling. It is not, however, adversarially perfect: an AI can satisfy the
 143 kernel by *changing the question*—adding the Axiom of Choice, weakening hypotheses, or exploiting
 144 a security vulnerability in Lean itself [20]—and the proof will still type-check [39]. To make gaming
 145 visible, every submission is scored on two axes: a binary *proved* flag (the file compiles under `lake`
 146 `build` and every original sorry has been discharged) and partial credit (fraction of original sorries
 147 discharged). We additionally record any axiom dependencies the proof introduces, since an AI can
 148 trivialize a goal by quietly importing strong axioms; we do not currently auto-fail on extra axioms,
 149 but report them so reviewers can flag suspicious solutions. For higher-stakes evaluations, submissions
 150 can additionally be re-checked through `leanprover/comparator` [30], which sandboxes the build
 151 and cross-validates the proof term against an independently-implemented Rust kernel. Lastly, a
 152 sufficiently motivated AI can still smuggle a vacuous proof past us when the original theorem
 153 statement is itself too weak—a limit shared by every benchmark whose scale exceeds full human
 154 review capacity.

155 Next, we explain how we scrape the PBT corpus and what the corpus looks like.

156 3 Property-Based Test Corpus (FVSpec:PBT)

157 We scrape real-world Python property-based tests from public GitHub repositories that depend on
 158 Hypothesis [38]. Our scraper operates in three stages. (1) *Discovery*. We crawl the Hypothesis
 159 dependency graph using PyGitHub [26] and persist 19,808 unique owner/repo pairs into a static
 160 snapshot. (2) *Licensing*. We skip repositories with an explicitly non-permissive license (e.g. AGPL,
 161 GPL, LGPL, MPL), and record the license string—including “unknown” for repositories with no
 162 detectable license—alongside the rest of each repository’s metadata. (3) *Extraction*. We shallow-clone
 163 each accepted repository, locate PBTs via `ripgrep`, and parse the surrounding Python to recover the
 164 transitive closure of locally-defined functions called therein.⁵

165 A 24-hour run produced 54,345 raw PBTs. The GitHub dependency graph surfaces many forks
 166 of the same upstream (e.g. hundreds of forks of `pytorch` carrying an identical Hypothesis suite),
 167 so we group repositories by base name, retain the most-starred “canonical” fork per group, and
 168 additionally drop tests whose whitespace-normalized SHA-256 already appears in the canonical set.
 169 After deduplication the working corpus is 11,039 PBTs across 333 repositories owned by 281 distinct
 170 GitHub users, spanning 303 distinct upstream project names.

171 3.1 Diversity

172 A perennial concern with scraped benchmarks is concentration: SWE-bench [28], for instance, draws
 173 all 2,294 of its task instances from just 12 repositories, with `django` alone supplying 37% and the top
 174 three (`django`, `sympy`, `scikit-learn`) supplying 64%. Our corpus is far more spread out—it draws
 175 from 333 repositories, no single one of which contributes more than 8.7% of the PBTs and whose top
 176 ten together contribute 58.5%—and most of those repositories are unstarred personal projects rather
 177 than curated showcase code (Table 2). The median PBT is just 13 LoC, but exercises around $3\times$ its
 178 own length in project code (see Figure 3).

⁵We also scrape TypeScript PBTs written using `fast-check` [18]; however, we chose to scope our Lean extraction in this work to just the Python PBTs due to token budget. The TypeScript PBTs are included in our dataset for future use.

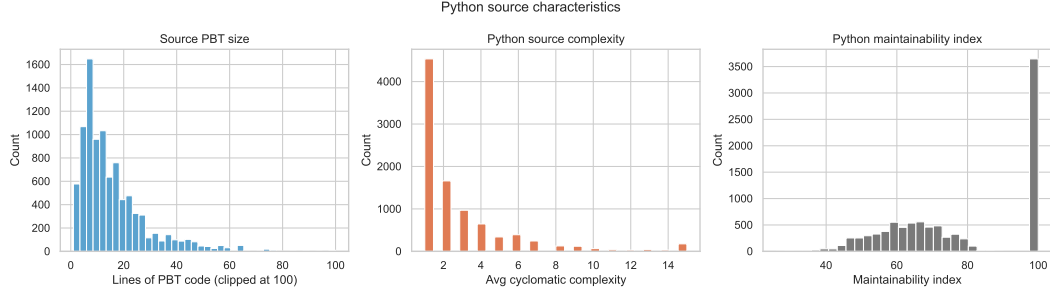


Figure 2: Code-quality metrics for the 11,039 deduplicated PBTs, computed by Radon [1]. *Left*: lines of code. *Center*: cyclomatic complexity—the number of linearly independent execution paths through the function, i.e. one plus the number of binary branch points [40]; higher values indicate more complex control flow. *Right*: maintainability index—a 0–100 composite of Halstead volume [21], cyclomatic complexity, and source lines, where higher is more maintainable; the spike at 100 is an artefact of Radon capping the score at that value.

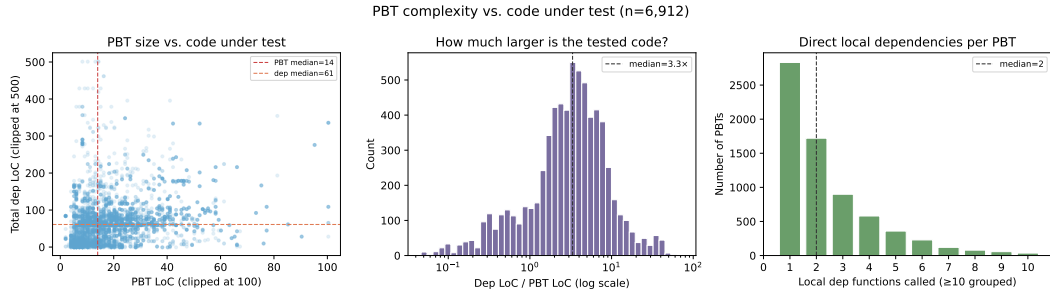


Figure 3: Three views of the relationship between PBT size and the size of the code being exercised, over the $n = 6,912$ post-deduplication PBTs whose local dependencies were recoverable. (The remaining 4,127 either exercise only standard-library or external code, or had dep extraction fail; the figure is therefore a subset of the full 11,039-PBT corpus.) (a) Per-PBT scatter of PBT LoC against the total LoC of the directly-called dependencies. (b) Histogram of the ratio between dependency LoC and PBT LoC (log scale); the median PBT exercises code $3.3\times$ its own length. (c) Histogram of the number of direct local function calls per PBT.

To illustrate the diversity of our dataset, consider the result of drawing ten repositories from it uniformly at random:⁶ `sec-parser` (SEC-filings parser), `whenever` (datetime library), `LDAR_Sim` (methane leak-detection simulator), `bowtie` (JSON-schema test runner), `Ivy-0cternships-ML` (ML-framework scaffolding), `aaanalysis` (amino-acid analysis library), `shrinkray` (test-case shrinker), `logot` (log-output assertion library), `tokenizations` (NLP tokenisation), and `onnx-embedding` (ONNX-backed embedding library). This is broadly representative of real-world software engineering, as opposed to purely academic or competitive programming.

Figure 2 summarizes three code-quality metrics across the corpus. The source tests span a wide range of complexity, from trivial single-assertion tests to multi-strategy tests exercising complex library APIs—diversity inherited directly from real-world engineering practice and not curated for difficulty.

3.2 Comparison with the Hypothesis Corpus

While we were finalizing this work, the Hypothesis maintainers released their own scraped corpus [14] (HC-2026). The two corpora target different downstream uses—HC-2026 is built to study Hypothesis *runtime* behavior (entropy consumption, predicate firing, coverage), while ours is built to feed a Lean formalization pipeline—which drives the design differences summarized in Table 3. A consequence of HC-2026’s runtime focus is that it filters out as “invalid” any repository whose PBTs may not

⁶Reproduce with `random.seed(13); random.sample([r for r in repos if r.stars >= 10], 10)` against the 333-repo deduplicated corpus; the star floor filters out personal forks.

Table 2: PBT corpus statistics.

Metric	Value
PBTs (raw scrape)	54,345
PBTs (post-dedup)	11,039
Unique repositories	333
Unique GitHub owners	281
Upstream project names	303
Top-1 / top-10 PBT share	8.7% / 58.5%
<i>Per-repo (min / med / max):</i>	
PBTs per repo	1 / 6 / 956
Stars	0 / 0 / 9,438
Forks	0 / 0 / 1,567
<i>Per-PBT (min / med / max):</i>	
PBT LoC	2 / 13 / 174
Direct fn deps	0 / 1 / 15

Table 3: Comparison with HC-2026 [14].

	FVSpec:PBT	HC-2026
Discovery	dep. graph	GitHub search
Raw PBTs	54,345	38,885
Tests (deduped)	11,039	23,139
Repos	333	1,504
Repo dedup	fork/name	is_fork
Test dedup	SHA-256	parametrization
License filter	no copyleft	none
Per-test data	source + metrics	runtime + coverage
Repo overlap (valid)	28 / 333	(8%)
Repo overlap (any)	111 / 333	(33%)

be directly runnable without some custom setup: code that depends on external services, API keys, environment variables, or live databases. Our pipeline imposes no such requirement, since we only need a PBT’s source code to translate it into Lean. Of our 333 repositories, 28 appear in HC-2026’s valid (executable) set and 111 (33%) appear in the pre-validity superset—meaning roughly 80 of our repositories were dropped by HC-2026 for being too operationally entangled to run, consistent with our goal of producing challenging and realistic FV problems. Future work could merge the two corpora to enlarge the input pool; we did not pursue this here because the present token budget does not exhaust even our own 11,039-PBT corpus.

Next, we describe the pipeline by which we transpile our PBT corpus into Lean FV challenges.

4 FVSpec:PBT → FV Pipeline

We lift each PBT in the Section 3 corpus (FVSpec:PBT) into a Lean (`Impl.lean`, `Spec.lean`) pair (Figure 5, bottom) in three phases: function discovery, agentic transpilation, and post-production. We call the resulting FV challenge corpus FVSpec:FV. The pipeline is modeled on FVAPPS [17] but starts from PBTs rather than programming puzzles, and adds an iterative LSP-repair loop.

Function discovery. For each PBT we use tree-sitter to extract the function under test (FUT) and the transitive closure of its locally-defined dependencies. For our running example `isort`, this is just the four-line `def isort` body itself: `isort` comes from `bisect`, which is external and not in scope for transitive recovery. Larger PBTs in the corpus pull in tens to hundreds of lines of internal helpers (see Figure 3). When discovery fails—typically due to metaclasses, decorators, or other dynamic dispatch—we emit a stub FUT with generic type parameters so the formalization agent still receives well-formed input.

Agentic transpilation. A single formalization agent translates each sample, producing both `Impl.lean` (a computable port of the FUT and its dependencies, no `sorry`) and `Spec.lean` (one theorem per Hypothesis assertion, with proofs left as `sorry`). This is exemplified for `isort` in Figure 1. After each generation step we type-check the output via the Lean LSP (over MCP) and `lake build`; on failure, the compiler diagnostics are fed back to the agent and a revised translation is requested. The loop terminates on success or after 16 tool-iterations.

Post-production. We first extract LLM turn counts from the raw `.eval` archives produced by the formalization agent, which runs as an Inspect AI [52] task. We then collapse runs into a single JSONL, deduplicating by sample id and tie-breaking with a quality score (structural faithfulness, theorem count, unit-tests, implementation success). Next, we re-run `lake build` on each retained sample and drop failures, which we classify into nine families: timeouts, well-founded recursion, unknown identifiers, failures to synthesize, type mismatches, syntax errors, missing functions, application errors, and declaration errors. We then run `radon` on the Python source and a custom AST analyzer on the Lean artifacts to extract line counts, theorem and `def` counts, residual `sorry/admits`, and axioms. Finally, conditioning on these metrics, we assign a binary easy/hard label using `claude-haiku-4-5` as judge, together with a calibrated confidence score and a short rationale.

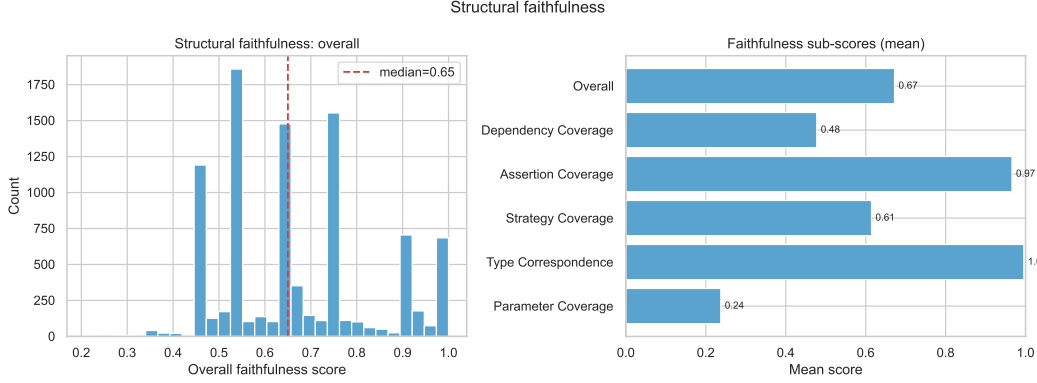


Figure 4: Structural-faithfulness scores across the published FVSpec:FV dataset. *Left*: histogram of per-sample overall scores (median 0.65; the multi-modal shape reflects the discrete denominators of the underlying ratios). *Right*: mean per-sub-metric scores. Type correspondence (1.00) and assertion coverage (0.97) are near-saturated—most translations preserve types and predicates—while parameter coverage (0.24) and dependency coverage (0.48) are the weak links, dragging the overall mean down to 0.67. The high variance in per-sample scores is exactly what the `is_canonical` flag exploits when multiple formalizations of the same PBT are available.

232 The structural-faithfulness score used in the merge tie-break is a fixed weighted average of five
 233 sub-scores, each measuring how much of a Python-side artifact survives translation:

$$S = 0.25 S_{\text{param}} + 0.25 S_{\text{type}} + 0.20 S_{\text{strat}} + 0.20 S_{\text{assert}} + 0.10 S_{\text{dep}} \quad (1)$$

234 where each sub-score is in $[0, 1]$: S_{param} (parameter coverage), S_{type} (type correspondence), S_{strat}
 235 (strategy coverage), S_{assert} (assertion coverage), and S_{dep} (dependency coverage). For `isort`, pa-
 236 rameter coverage rewards the Lean theorem taking a single `List Int` matching Python’s `list`; type
 237 correspondence verifies that Python’s `list[int]` maps to `List Int`; strategy coverage tracks that
 238 `st.lists(st.integers())` ranges over arbitrary integer lists, with no missing bounds in the Lean
 239 version; assertion coverage rewards both Python assertions producing a corresponding Lean theorem;
 240 and dependency coverage rewards every named helper recovered in function discovery surviving into
 241 `Impl.lean`. Sub-scores ship alongside the overall score so consumers can re-weight or filter.

242 4.1 Pipeline yield

243 The published FVSpec:FV dataset contains **9,415 samples** drawn from **2,772 distinct PBTs** (Figure 5,
 244 bottom right): formalization succeeded for 2,772 of 11,039 input PBTs (25%); the rest were dropped
 245 by `lake build`. The mean sample carries 8 theorems (median 6, max 66) and median structural
 246 faithfulness 0.65 (Figure 4). A complete Lean implementation was generated for 59% of samples;
 247 the remaining 41% ship against a stub `Impl.lean`, so the theorem statement is well-typed but the
 248 body it constrains is opaque. The Claude Haiku grader splits the dataset 62%/38% hard/easy. Where
 249 a PBT yields multiple compiling formalizations (mean 3.4 formalizations per PBT, range 1–15), all
 250 are kept; the `is_canonical` field marks the highest-quality version per PBT.

251 5 Benchmark Characterization

252 The released dataset contains 9,415 Lean 4 formalization challenges derived from 2,772 unique Python
 253 property-based tests, produced by two pipeline runs (`feb03`: 5,979 samples, Claude Sonnet; `apr08`:
 254 3,436 samples, Claude Sonnet 4.6). Each unique PBT has between 1 and 15 formalizations. Among
 255 PBTs with multiple formalizations, 65% have no single dominating attempt: for each formalization in
 256 the group there exists another that scores higher on at least one sub-metric of Equation 1. In practice
 257 this means, for example, that the attempt with the best assertion coverage may have worse parameter
 258 coverage than a sibling; keeping all formalizations and flagging the best one with `is_canonical`
 259 therefore captures strictly more information than retaining only the top-scoring attempt.

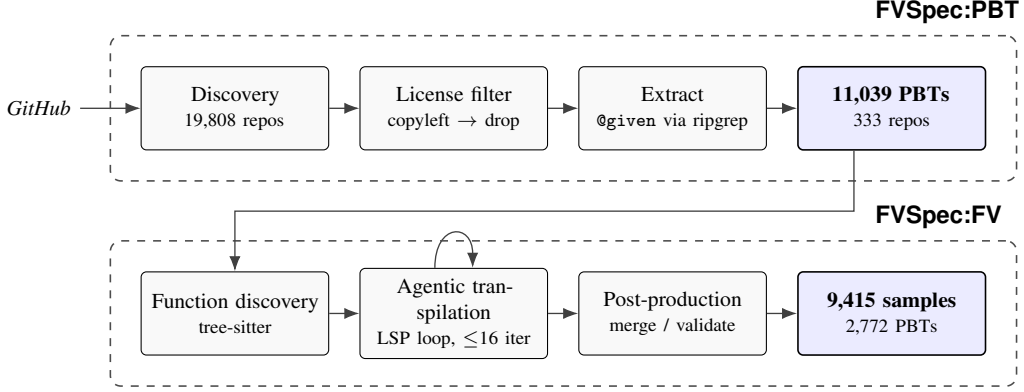


Figure 5: The full FVSpec generation pipeline (FVC = FV challenge). *Top* (FVSpec:PBT): the input PBT corpus is scraped from public GitHub repositories (Section 3). *Bottom* (FVSpec:FV): each PBT is lifted into a Lean (Impl, Spec) pair via tree-sitter-based function discovery, an agentic transpilation step (with the LSP-repair self-loop), and a deterministic post-production stage that validates, deduplicates, and grades the output; 11,039 PBTs enter and 9,415 FVCs (from 2,772 distinct PBTs) emerge.

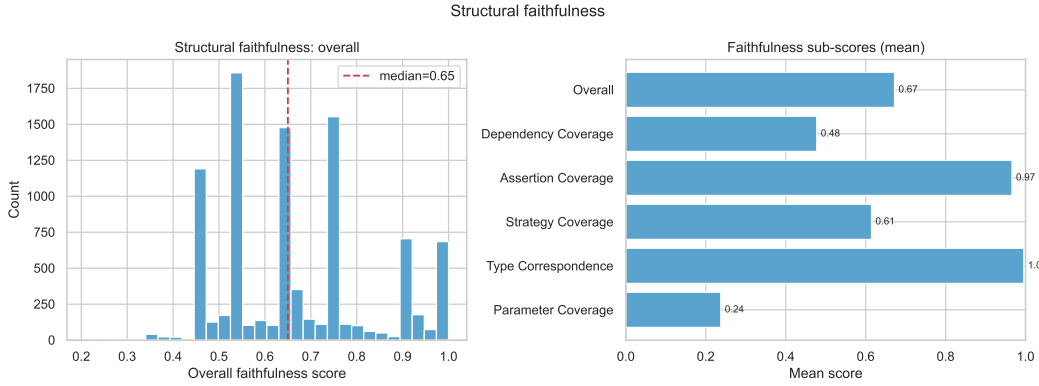


Figure 6: Structural faithfulness: distribution of the overall composite score (left) and mean sub-scores (right).

260 5.1 Translation Quality

261 We evaluate translation quality using *structural faithfulness* (Equation 1), a composite metric defined
 262 in Section 4. Figure 6 shows the distribution of the overall score and the mean of each sub-score
 263 across the dataset. The overall faithfulness distribution exhibits a clear mode above 0.5, indicating
 264 that the majority of translations preserve the essential structure of the source PBT, though a long tail
 265 of lower-quality translations remains.

266 In 59% of samples the agent generated a complete Lean implementation (Impl.lean contains a
 267 computable definition, not just a stub); the remaining 41% use a signature-only placeholder. Structural
 268 faithfulness is comparable across both groups (mean 0.67 vs. 0.68), confirming that specification
 269 quality is largely independent of whether the implementation was recovered.

270 The generated Lean artifacts span a wide range of complexity (Figure 7): the median sample contains
 271 multiple theorems each requiring an independent proof, and the sorry count tracks the theorem
 272 count closely, as expected from our pipeline design.

273 Per-sample pipeline cost is right-skewed (Figure 8): most samples compile within a few agent turns,
 274 but a tail of difficult translations requires many repair iterations before the Lean LSP reports success
 275 or the budget is exhausted.

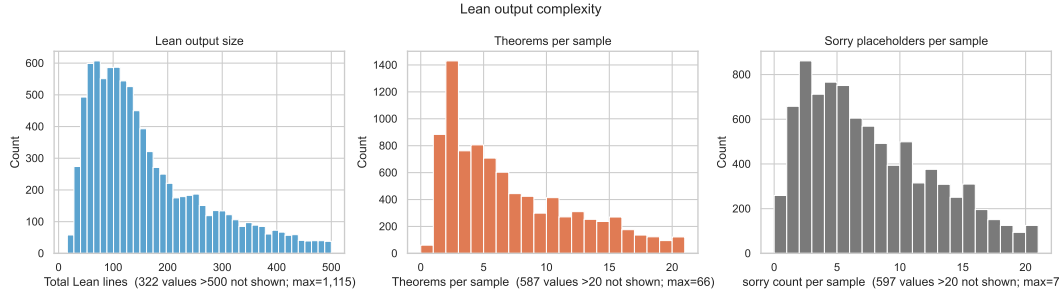


Figure 7: Lean output complexity: total lines (left), theorems per sample (center), and sorry placeholders per sample (right). Tail counts and maxima shown in axis labels.

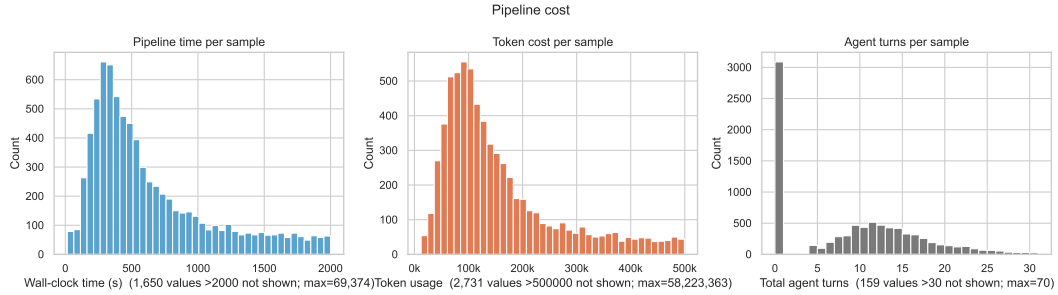


Figure 8: Pipeline cost per sample: wall-clock time (left), token usage (center), and agent turns (right). Tail counts and maxima shown in axis labels.

276 5.2 Difficulty Grading

277 Each sample is graded for proof difficulty by Claude Haiku 4.5, which assigns a binary label (*easy* or
 278 *hard*) along with a confidence score. Figure 9 shows the resulting split and the distribution of grader
 279 confidence. The benchmark contains a substantial proportion of both easy and hard problems, and
 280 the grader expresses high confidence for the majority of assessments. Difficulty is estimated here as a
 281 *prediction*, where we ask the grader to predict whether or not a language model agent would be able
 282 to complete the proofs successfully (considering the problem "hard" if haiku thinks not, and "easy" if
 283 haiku thinks yes).

284 Figure 10 examines the relationship between difficulty and two output characteristics: structural
 285 faithfulness and Lean output size. Hard problems tend to produce longer Lean outputs, consistent
 286 with the intuition that more complex source PBTs lead to more elaborate formalizations and more
 287 challenging proofs.

288 5.3 Baselines

289 We evaluate two frontier models (Claude Sonnet 4.6 and GPT 5.4) on 100 randomly sampled easy
 290 problems and 100 hard problems (difficulty as defined in Section 5). Each model has access to the
 291 Lean LSP via MCP tools and is scored on two metrics: a binary *proved* flag (zero *sorry* remaining
 292 and *lake build* succeeds) and *partial credit* (fraction of *sorry* placeholders removed, clamped to
 293 $[0, 1]$).

294 6 Threats to Validity

295 The most consequential threat to FVSpec’s validity is whether theorems stated over uninterpreted
 296 axioms actually mean anything about the original Python. We address this head-on in Section 6.1,
 297 then catalog the remaining threats.

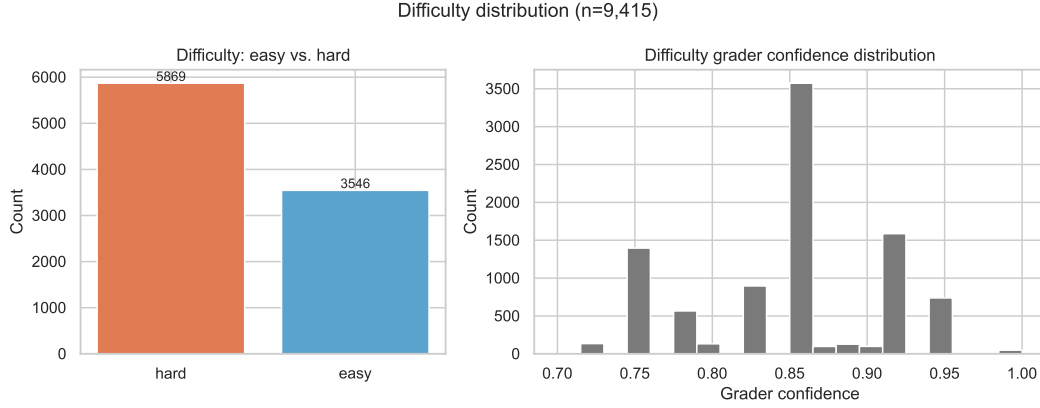


Figure 9: Difficulty distribution: easy vs. hard counts (left) and grader confidence distribution (right). n shown in title.

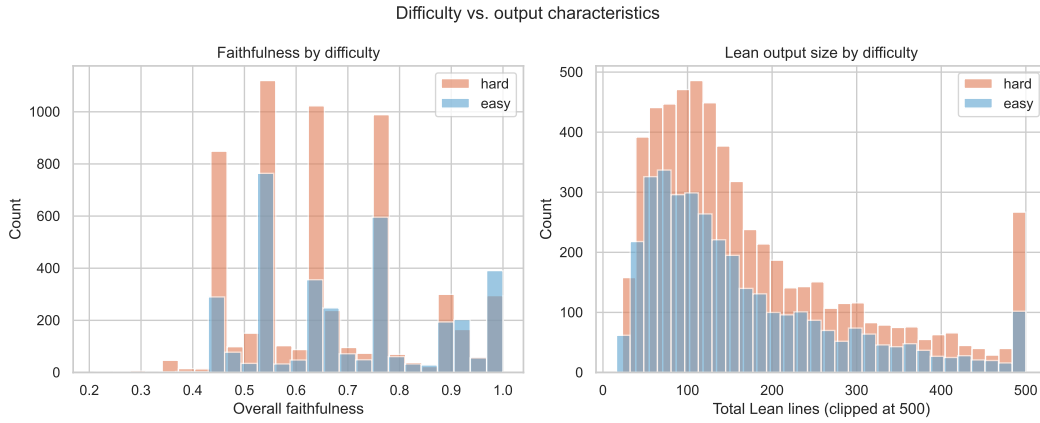


Figure 10: Difficulty vs. output characteristics: overall faithfulness (left) and total Lean lines (right), stratified by difficulty.

6.1 Axiomatized interfaces for effectful code

The `isort` running example from Section 2 is deliberately pure. Most PBTs in our corpus are not: production code routinely touches Redis, sockets, the filesystem, the wall clock, webhook queues, and so forth. A natural objection is that any Lean theorem extracted from such a PBT cannot say anything meaningful about the live system.

The formalization agent handles this by treating the effectful world as an *uninterpreted interface*. Parts of the source essential to the property—the entities and operators it constrains—are translated into Lean structures and inductive types. Parts that are pure plumbing—the database, the queue, the system clock—are introduced as opaque axiom declarations and threaded through the theorem as parameters. The resulting theorem holds *over all interpretations* of those axioms, which is exactly the contract the original PBT was checking against the live environment.

A canonical instance is `test_workflow_job_time_to_start` from `scality/runner-manager`⁷, a GitHub Actions self-hosted runner controller. The Python PBT initializes Redis-backed model classes, runs an ORM migrator, calls `datetime.now()` to fabricate two timestamps straddling a runner-startup timeout, enqueues a webhook through a background job queue, and asserts on the resulting runner count. The FVSpec translation introduces seventeen axioms—including `Redis`, `Queue`, `State`, `State.after_enqueue`, and `get_runners`—and states the contract purely in terms of these uninterpreted operators:

⁷<https://github.com/scality/runner-manager>

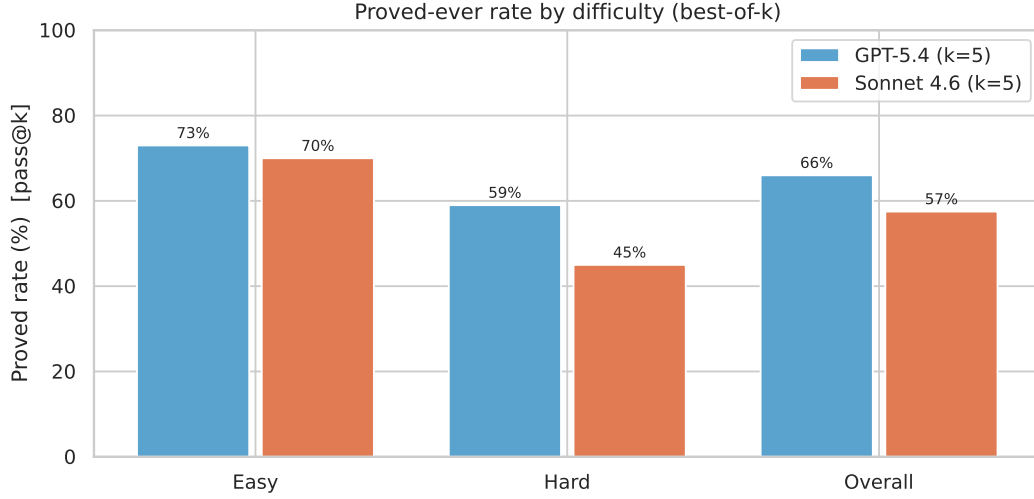


Figure 11: baseline prove rate

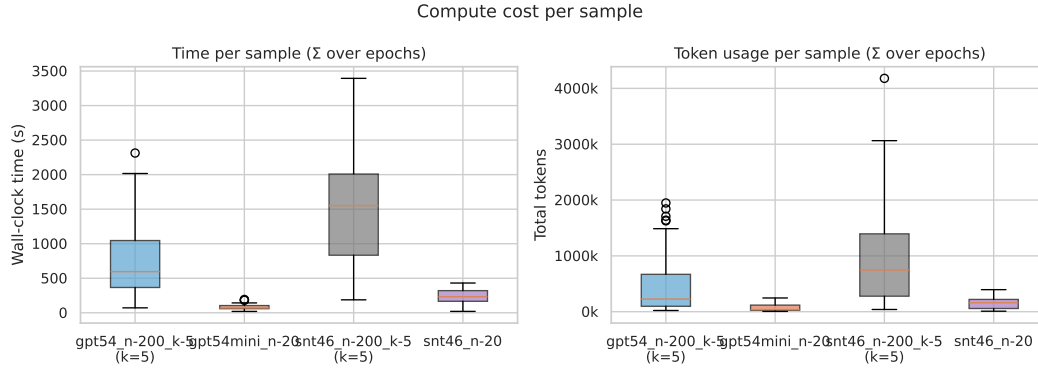


Figure 12: Compute cost per sample for each baseline run: wall-clock time (left) and token usage (right), summed across all attempts for $k > 1$ runs. Box plots show median, IQR, and $1.5 \times \text{IQR}$ whiskers.

```

316 theorem create_runner_when_above_timeout
317   (initial_state : State) (state_after_enqueue : State)
318   (runner_group : RunnerGroup) (webhook_updated : WorkflowJobEvent)
319   (settings : ExtendedSettings) (created_at started_at : Time)
320   (h_initial_count : (get_runners runner_group initial_state).length = 1)
321   (h_below_max : 1 < runner_group.max)
322   (h_above_timeout : started_at - created_at > settings.timeout_runner)
323   (h_enqueue : state_after_enqueue =
324     State.after_enqueue initial_state runner_group webhook_updated settings)
325   : (get_runners runner_group state_after_enqueue).length = 2 := sorry

```

326 A model proving `create_runner_when_above_timeout` is reasoning about a scheduler’s state-
 327 transition semantics under a timeout precondition; the fact that the production scheduler talks to
 328 Redis is no more load-bearing in the theorem than the choice of register allocator is in a correctness
 329 proof of a sorting algorithm.

330 6.2 Where axiomatization breaks down

331 The axiomatization move is not a universal solvent.

332 *Properties about the effect itself.* If the property is fundamentally about timing, memory, or-
 333 dering of side effects, or wall-clock latency, an uninterpreted axiom cannot witness it. `assert`

334 `response_time < 100ms` cannot become a meaningful Lean theorem because the relevant seman-
335 tics is precisely what the axiomatization discards.

336 *Effectful code disguised as pure code.* The agent identifies effectful boundaries via type signatures
337 and call patterns. When a function has a pure signature but performs side effects internally (e.g.,
338 a logger called via a global, an `lru_cache`-style hidden state), the agent will inline it as if pure.
339 The resulting theorem may compile and even be true, but its proof obligation does not constrain the
340 original side-effecting behavior.

341 *Axioms that misrepresent the API contract.* `axiom Redis` : Type captures *some* Redis but not the
342 real one. The agent does not encode Redis’s consistency guarantees, ordering, or failure modes. A
343 theorem proven over the axiomatized interface may not transfer to the live system if the production
344 code relies on those omitted guarantees.

345 6.3 Other threats

346 *Translation faithfulness.* Specifications are translated rather than hand-written, so some may not
347 faithfully represent the original PBT. Our structural-faithfulness score (Section 4) is a heuristic: a
348 high score does not guarantee semantic equivalence, and a low score does not necessarily indicate a
349 useless problem.

350 *LLM grader.* Difficulty labels come from `claude-haiku-4-5`, itself a model whose judgments are
351 imperfect and uncalibrated against human expert performance. We expose the grader’s confidence
352 and rationale per sample so consumers can audit or override the labels.

353 *Stub-implementation tail.* A complete Lean implementation is generated for 59% of samples (Sec-
354 tion 4); the remaining 41% ship against a stub, weakening the proof obligation: the theorem is
355 well-typed but the body it constrains is opaque. These samples are tagged for easy filtering.

356 *License-detection edge cases.* Our scraper (Section 3) skips repositories with explicitly non-permissive
357 licenses but accepts repositories where GitHub cannot detect a license at all. Most such cases are
358 forks of permissively-licensed upstreams (e.g., `pytorch`, `ivy`) where the `LICENSE` file was modified
359 or moved during forking, but we cannot guarantee this for every retained sample.

360 *No human baselines.* We do not measure human expert performance on FVSpec problems and
361 therefore cannot calibrate the dataset’s difficulty distribution against the proof effort required of
362 skilled humans.

363 7 Related Work

364 Our work extends FVAPPS by Dougherty and Mehta [17], which established AI-driven translation
365 of coding problems into Lean theorems. In FVAPPS, source programs were taken from the APPS
366 benchmark [22], translated to PBTs, then lifted to theorems. We apply the same process at scale to
367 PBTs found in the wild, inferring specifications from natural language and source code.

368 *Lean verification benchmarks.* A growing number of benchmarks target Lean specifically. `miniCode-`
369 `Props` [6] provides 177 program specifications translated from TIP (Tons of Inductive Programs),
370 finding that current LLM provers fail on medium and hard properties. `CLEVER` [49] offers 161
371 curated problems sourced from HumanEval, testing end-to-end verified code generation. `Verina` [55]
372 covers all three foundational tasks—code generation, specification generation, and proof generation—
373 across 189 manually curated coding tasks. `VeriBench` [47] translates Python code with documentation
374 into Lean implementations, specifications, and unit tests (113 tasks). The `Vericoding Benchmark` [8]
375 supports Lean, Rust/Verus, and Dafny, reporting success rates from 27% (Lean) to 82% (Dafny). `Veri-`
376 `EquivBench` [37] uses LeetCode transformations to evaluate verifiable agents on novel data without
377 contamination. These benchmarks are valuable but share a common limitation: their specifications
378 derive from synthetic coding problems or pedagogical exercises, not from properties written by
379 practicing engineers.

380 *Other verification tool benchmarks.* `DafnyBench` [34] provides Dafny verification tasks. `VerusBench`
381 [10] collects 150 proof tasks spanning Dafny, SV-COMP, and Verus. `SV-COMP` [5] is the standard
382 competition for software verifiers, with over 33,000 C tasks, but targets automated verification rather

than interactive theorem proving. InvBench [54] focuses specifically on loop invariant synthesis. Chakraborty et al. [9] explore neural synthesis for SMT-assisted proof-oriented programming in F*.

Interactive theorem proving benchmarks. LeanDojo Benchmark 4 [23] provides 122,517 theorems from Mathlib4. CoqStoq [50] collects 196,929 theorems from 2,226 GitHub Coq repositories. These are primarily mathematical rather than software-oriented, and their theorems exist in the training data of most frontier models.

Mathematics benchmarks. FrontierMath [19] evaluates advanced mathematical reasoning. While formal verification and mathematics share proof infrastructure, the skills required differ substantially: software verification demands modelling of program semantics, library APIs, and computational behavior, rather than abstract mathematical reasoning.

FVSpec differs from each existing benchmark in at least one of the following three ways. First, our specifications originate from real-world property-based tests written by practicing engineers, not synthetic problems or pedagogical exercises. Second, the source PBTs were never formally verified, so most theorems in our benchmark will not appear in any model’s training data. Third, we operate at a larger scale than existing Lean verification benchmarks, with thousands rather than hundreds of problems. Our work is motivated by the broader context of AI safety proposals premised on AI-driven formal verification, including the Guaranteed Safe AI agenda [11].

8 Conclusion

We present FVSpec, a benchmark of 9,415 Lean 4 formal verification challenges derived from 11,039 real-world Python property-based tests scraped from 333 open-source repositories. Unlike prior FV benchmarks—which draw from expert-written ITP code, competitive programming problems, or curated verification exercises—FVSpec sources its specifications directly from practicing engineers who had no formal verification intent, placing its problems genuinely out of distribution relative to models’ training data. Our agentic formalization pipeline produced 9,415 challenges from 2,772 of those PBTs on a single pass through the corpus (25% lake-build success rate), with median structural faithfulness 0.65 and 62% of challenges classified as hard by a calibrated difficulty predictor. Baseline evaluations of two frontier models confirm that the benchmark is far from saturated, particularly on hard problems, leaving substantial room for progress on AI-assisted formal verification of ordinary software. Natural extensions include incorporating PBTs from other languages (Haskell/QuickCheck, C++/RapidCheck, TypeScript/fast-check) and other ITPs (Coq, Isabelle, F*, Verus), merging with the HC-2026 corpus to add runtime context, and using disproved properties as seeds for program-repair challenges—bugs the original developer’s test harness missed, now made precise in Lean.

References

- [1] M. Attanasio. Radon: Python code metrics. <https://radon.readthedocs.io/en/latest/>. Accessed 2025.
- [2] Z. Azerbayev, B. Piotrowski, H. Schoelkopf, E. W. Ayers, D. Radev, and J. Avigad. Proofnet: Autoformalizing and formally proving undergraduate-level mathematics. *arXiv preprint arXiv:2302.12433*, 2023.
- [3] K. Bansal, S. Loos, M. Rabe, C. Szegedy, and S. Wilcox. Holist: An environment for machine learning of higher order logic theorem proving. In *International conference on machine learning*, pages 454–463. PMLR, 2019.
- [4] S. Barkallah, S. Daruru, B. Miranda, L. Aniva, A. Nie, and S. Koyejo. VeriBench-FTP: A formal theorem proving benchmark in Lean 4 for code verification. In *The 5th Workshop on Mathematical Reasoning and AI at NeurIPS 2025*, 2025.
- [5] D. Beyer. State of the art in software verification and witness validation: SV-COMP 2025. In *Proceedings of the 31st International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, 2025.
- [6] D. Brandfonbrener, L. Beurerkellner, D. Romero, N. Amin, and M. Vechev. miniCodeProps: a minimal benchmark for proving code properties. *arXiv preprint arXiv:2406.11915*, 2024.

- [7] S. Buro, R. Crole, and I. Mastroeni. Equational logic and categorical semantics for multi-languages. *Electronic Notes in Theoretical Computer Science*, 352:79–103, 2020.
- [8] S. Bursuc, T. Ehrenborg, S. Lin, L. Astefanoaei, I. E. Chiosa, J. Kukovec, A. Singh, O. Butterley, A. Bizid, Q. Dougherty, et al. A benchmark for vericoding: formally verified program synthesis. *arXiv preprint arXiv:2509.22908*, 2025.
- [9] S. Chakraborty, G. Ebner, S. Bhat, S. Fakhoury, S. Fatima, S. Lahiri, and N. Swamy. Towards neural synthesis for SMT-assisted proof-oriented programming. In *Proceedings of the 47th International Conference on Software Engineering (ICSE)*, 2025.
- [10] H. Chen et al. VerusBench: A benchmark for automated verification of Rust with Verus. *arXiv preprint arXiv:2409.13082*, 2024.
- [11] D. Dalrymple, J. Skalse, Y. Bengio, S. Russell, M. Tegmark, S. Seshia, S. Omohundro, C. Szegedy, B. Goldhaber, N. Ammann, A. Abate, J. Halpern, C. Barrett, D. Zhao, T. Zhi-Xuan, J. Wing, and J. Tenenbaum. Towards guaranteed safe AI: A framework for ensuring robust and reliable AI systems. *arXiv preprint arXiv:2405.06624*, 2024.
- [12] N. A. David Dalrymple. Safeguarded ai. <https://aria.org.uk/opportunity-spaces/mathematics-for-safe-ai/safeguarded-ai/>, 2025. Accessed 25 April 2026.
- [13] A. de Almeida Borges, A. Casanueva Artís, J.-R. Falleri, E. J. Gallego Arias, É. Martin-Dorel, K. Palmkog, A. Serebrenik, and T. Zimmermann. Lessons for interactive theorem proving researchers from a survey of coq users. *Journal of Automated Reasoning*, 69(1):8, 2025.
- [14] L. DeVoe. The Hypothesis corpus. <https://hypothesis.works/articles/hypothesis-corpus/>, 2026. Accessed April 2026.
- [15] P. C. Dillinger, P. Manolios, D. Vroon, and J. S. Moore. Acl2s: “the acl2 sedan”. *Electronic Notes in Theoretical Computer Science*, 174(2):3–18, 2007.
- [16] M. Dodds. Formally verifying industry cryptography. *IEEE Security & Privacy*, 20(3):65–70, 2022. doi: 10.1109/MSEC.2022.3153035.
- [17] Q. Dougherty and R. Mehta. Proving the coding interview: A benchmark for formally verified code generation. In *Proceedings of the LLM4Code Workshop at the 47th International Conference on Software Engineering (ICSE)*, Ottawa, Ontario, Canada, 2025. doi: 10.48550/arXiv.2502.05714.
- [18] N. Dubien. fast-check: Property-based testing framework for JavaScript/TypeScript. <https://github.com/dubzzz/fast-check>, 2024. Accessed April 2026.
- [19] E. Glazer, E. Erdil, T. Besiroglu, D. Chicharro, E. Chen, A. Gunning, C. Falkman Olsson, J.-S. Denain, A. Ho, E. de Oliveira Santos, O. Järvinen, M. Barnett, R. Sandler, M. Vrzala, J. Sevilla, Q. Ren, E. Pratt, L. Levine, G. Barkley, N. Stewart, B. Grechuk, T. Grechuk, S. Varma Enugandla, and M. Wildon. FrontierMath: A benchmark for evaluating advanced mathematical reasoning in AI. *arXiv preprint arXiv:2411.04872*, 2024.
- [20] K. Gopinathan. Lean proved this program was correct; then I found a bug. <https://kirancodes.me/posts/log-who-watches-the-watchers.html>, Apr. 2026. Blog post; reports a heap buffer overflow in `lean_alloc_sarray` in the Lean 4 runtime, found by AFL++ fuzzing of the verified `lean-zip` library. Accessed 26 April 2026.
- [21] M. H. Halstead. *Elements of Software Science (Operating and Programming Systems Series)*. Elsevier Science Inc., 1977.
- [22] D. Hendrycks, S. Basart, S. Kadavath, M. Mazeika, A. Arora, E. Guo, C. Burns, S. Puranik, H. He, D. Song, and J. Steinhardt. Measuring coding challenge competence with APPS. In *Proceedings of the 35th Conference on Neural Information Processing Systems (NeurIPS)*, 2021.

- [23] R. Hsiang, W. Adkisson, R. J. George, and A. Anandkumar. LeanDojo-v2: A comprehensive library for AI-assisted theorem proving in Lean. In *The 5th Workshop on Mathematical Reasoning and AI at NeurIPS 2025*, 2025.
- [24] J. Hu, T. Zhu, and S. Welleck. minictx: Neural theorem proving with (long-) contexts. *arXiv preprint arXiv:2408.03350*, 2024.
- [25] Y. Huang, X. Lin, Z. Liu, Q. Cao, H. Xin, H. Wang, Z. Li, L. Song, and X. Liang. Mustard: Mastering uniform synthesis of theorem and proof data. *arXiv preprint arXiv:2402.08957*, 2024.
- [26] V. Jacques, L. Wan, S. Kowalik, and E. Minack. PyGitHub. <https://github.com/PyGithub/PyGithub>, 2026. Accessed 26 April 2026.
- [27] J. Jiang, W. He, Y. Wang, G. Gao, Y. Hu, J. Wang, N. Guan, P. Wu, C. Dai, L. Xiao, and B. Dong. FATE: A formal benchmark series for frontier algebra of multiple difficulty levels. *arXiv preprint arXiv:2511.02872*, 2025.
- [28] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2024.
- [29] G. Klein, K. Elphinstone, G. Heiser, J. Andronick, D. Cock, P. Derrin, D. Elkaduwe, K. Engelhardt, R. Kolanski, M. Norrish, et al. sel4: Formal verification of an os kernel. In *Proceedings of the ACM SIGOPS 22nd symposium on Operating systems principles*, pages 207–220, 2009.
- [30] Lean FRO. Comparator: A trustworthy judge for Lean proofs. <https://github.com/leanprover/comparator>, 2026. Sandboxes the build and replays proof terms, optionally cross-checking against an independently-implemented Rust kernel (nanoda). Accessed 26 April 2026.
- [31] X. Leroy, S. Blazy, D. Kästner, B. Schommer, M. Pister, and C. Ferdinand. Compcert-a formally verified optimizing compiler. In *ERTS 2016: Embedded Real Time Software and Systems, 8th European Congress*, 2016.
- [32] S. Li, W. Lu, X. Shi, K. Weng, H. Sun, M. Yu, T. Zhang, G. Yu, H. Liu, and L. Du. MSC-180: A benchmark for automated formal theorem proving from mathematical subject classification. *arXiv preprint arXiv:2512.18256*, 2025.
- [33] C. Liu, J. Shen, H. Xin, Z. Liu, Y. Yuan, H. Wang, W. Ju, C. Zheng, Y. Yin, L. Li, et al. Fimo: A challenge formal dataset for automated theorem proving. *arXiv preprint arXiv:2309.04295*, 2023.
- [34] C. Loughridge, Q. Sun, S. Ahrenbach, F. Cassano, C. Sun, Y. Sheng, A. Mudide, M. R. H. Misu, N. Amin, and M. Tegmark. DafnyBench: A benchmark for formal software verification. *arXiv preprint arXiv:2406.08467*, 2024.
- [35] J. Lu, Y. Wan, Z. Liu, Y. Huang, J. Xiong, C. Liu, J. Shen, H. Jin, J. Zhang, H. Wang, et al. Process-driven autoformalization in lean 4. *arXiv preprint arXiv:2406.01940*, 2024.
- [36] P. Lu, J. Sheng, L. Lyu, J. Jin, T. Xia, A. Gu, and J. Zou. Solving inequality proofs with large language models. In *The 39th Conference on Neural Information Processing Systems (NeurIPS)*, 2025.
- [37] X. Ma et al. VeriEquivBench: Evaluating verifiable agents on novel data without contamination. *arXiv preprint arXiv:2510.06296*, 2025.
- [38] D. R. MacIver, Z. Hatfield-Dodds, and many other contributors. Hypothesis: A new approach to property-based testing. *Journal of Open Source Software*, 4(43):1891, 2019. doi: 10.21105/joss.01891.
- [39] Q. D. Max von Hippel. Lies, damned lies, and proofs: Formal methods are not slopless. <https://www.lesswrong.com/posts/rhAPh3YzhPoBNpgHg/lies-damned-lies-and-proofs-formal-methods-are-not-slopless>, Jan. 2026. Accessed 26 April 2026.

- [40] T. J. McCabe. A complexity measure. *IEEE Transactions on Software Engineering*, (4): 308–320, 1976.
- [41] E. Miyazono. Formally scalable AI oversight through specifications. <https://foresight.org/resource/evan-miyazono-formally-scalable-ai-oversight-through-specifications/>, 2024. Accessed 25 April 2026.
- [42] J. T. Perconti and A. Ahmed. Verifying an open compiler using multi-language semantics. In *European Symposium on Programming Languages and Systems*, pages 128–148. Springer, 2014.
- [43] A. Poiroux, G. Weiss, V. Kunčák, and A. Bosselut. Reliable evaluation and benchmarks for statement autoformalization. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 17958–17980, 2025.
- [44] J. Rasmussen. Risk management in a dynamic society: a modelling problem. *Safety science*, 27 (2-3):183–213, 1997.
- [45] Z. Ren, Z. Shao, J. Song, H. Xin, H. Wang, W. Zhao, L. Zhang, Z. Fu, Q. Zhu, D. Yang, et al. Deepseek-prover-v2: Advancing formal mathematical reasoning via reinforcement learning for subgoal decomposition. *arXiv preprint arXiv:2504.21801*, 2025.
- [46] C. Song, Z. Wang, F. Pu, H. Wang, X. Lin, J. Liu, J. Li, and Z. Liu. LeanGeo: Formalizing computational geometry problems in Lean. *arXiv preprint arXiv:2508.14644v1*, 2025.
- [47] K. Stechly et al. VeriBench: Benchmarking end-to-end formal verification. In *AI4Math Workshop at ICML*, 2025. URL <https://openreview.net/forum?id=rWkGFmnSN1>.
- [48] R. S. Team. 2025 state of rust survey results. <https://blog.rust-lang.org/2026/03/02/2025-State-Of-Rust-Survey-results/>, March 2026. Accessed 25 April 2026.
- [49] A. Thakur, J. Lee, G. Tsoukalas, M. Sistla, M. Zhao, S. Zetsche, G. Durrett, Y. Yue, and S. Chaudhuri. Clever: A curated benchmark for formally verified code generation. *arXiv preprint arXiv:2505.13938*, 2025.
- [50] K. Thompson, N. Saavedra, P. Carrott, K. Fisher, A. Sanchez-Stern, Y. Brun, J. F. Ferreira, S. Lerner, and E. First. Rango: Adaptive retrieval-augmented proving for automated software verification. *arXiv preprint arXiv:2412.14063*, 2024.
- [51] G. Tsoukalas, J. Lee, J. Jennings, J. Xin, M. Ding, M. Jennings, A. Thakur, and S. Chaudhuri. Putnambench: Evaluating neural theorem-provers on the putnam mathematical competition. *Advances in Neural Information Processing Systems*, 37:11545–11569, 2024.
- [52] UK AI Security Institute. Inspect AI: Framework for large language model evaluations, 2024. URL https://github.com/UKGovernmentBEIS/inspect_ai.
- [53] M. von Hippel. <https://www.lesswrong.com/posts/SfhFh9Hfm6JYvzbby/the-scalable-formal-oversight-research-program>, 2026. Accessed 25 April 2026.
- [54] T.-Y. Wu et al. InvBench: A benchmark for loop invariant synthesis. *arXiv preprint arXiv:2509.21629*, 2025.
- [55] W.-D. Wu, K. Wang, R. Mangal, and I. Dillig. Verina: Benchmarking formally verified code generation. *arXiv preprint arXiv:2505.23135*, 2025.
- [56] Z. Wu, J. Wang, D. Lin, and K. Chen. Lean-github: Compiling github lean repositories for a versatile Lean prover. *arXiv preprint arXiv:2407.17227*, 2024.
- [57] H. Ying, Z. Wu, Y. Geng, J. Wang, D. Lin, and K. Chen. Lean workbook: A large-scale lean problem set formalized from natural language math problems. *Advances in Neural Information Processing Systems*, 37:105848–105863, 2024.

- 572 [58] Z. Yu, R. Peng, K. Ding, Y. Li, Z. Peng, M. Liu, Y. Zhang, Z. Yuan, H. Xin, W. Huang, et al.
573 Formalmath: Benchmarking formal mathematical reasoning of large language models. *arXiv*
574 *preprint arXiv:2505.02735*, 2025.
- 575 [59] K. Zheng, J. M. Han, and S. Polu. Minif2f: a cross-system benchmark for formal olympiad-level
576 mathematics. *arXiv preprint arXiv:2109.00110*, 2021.